Architectural Paradigms for Agentic Developer Documentation: Evaluating Context7, Code Mode MCP, Agentic CLIs, and Hybrid Retrieval Systems

1. Executive Summary and the Physics of Agent Context

Modern software engineering agents suffer from a structural mismatch between static model pre-training weights and rapidly evolving library ecosystems[1]. When an agent interacts with fast-moving frameworks such as Next.js, Tailwind, Zod, or React Query, reliance on parametric memory results in hallucinated API signatures, deprecated configuration patterns, and broken builds[1]. While injecting live official documentation into the inference loop resolves knowledge staleness, the mechanism chosen to deliver that documentation dictates the economic viability, latency, and accuracy of the entire agentic harness[4].

Empirical evaluations across production agentic workflows demonstrate that the industry debate framing the Model Context Protocol (MCP) against Command-Line Interfaces (CLIs) misidentifies the underlying bottleneck[6]. MCP functions strictly as a transport and schema primitive; the decisive variables governing token consumption and task completion are upfront schema bloat, per-turn transcript accumulation, and multi-step orchestration topology[5]. Connecting a standard multi-tool MCP server such as the official GitHub MCP server injects between 43 and 93 tool definitions directly into the system prompt prefix, consuming 46,000 to 55,000 tokens before a single user prompt is processed[5]. Conversely, while a local CLI consumes zero upfront schema tokens by relying on parametric shell knowledge or compact skill files, naive CLI usage degrades on complex, chained workflows due to unstructured text parsing and repetitive multi-turn inference round-trips[5].

For a systems architect designing a free, high-efficiency alternative to commercial documentation servers like Context7 or Ref, the optimal architecture is neither a naive schema-heavy MCP server nor an unassisted shell script[6]. Instead, production evidence points toward a multi-surface, local-first hybrid retrieval engine backed by embedded vector and full-text search (sqlite-vec and SQLite FTS5), exposed simultaneously via an Agentic CLI with progressive disclosure and a Code Mode or Deferred-Loading MCP interface[6].

2. Internal Architecture and Economics of the Context7 Ecosystem

Under the Hood of Context7 MCP and the ctx7 CLI

Developed and maintained by Upstash, Context7 (@upstash/context7-mcp) was architected to

eliminate API hallucinations by intercepting coding prompts, resolving package identities, and injecting version-specific documentation and code excerpts directly into the active context window[1]. Rather than exposing dozens of granular API endpoints that cause schema bloat, Context7 restricts its MCP surface area to two primary tools that execute a sequential resolution-and-retrieval workflow[4]. The first tool, resolve-library-id, accepts a required libraryName string (such as "next.js" or "supabase") alongside a required query string describing the coding task[4]. The backend performs semantic and lexical ranking against Upstash's centralized catalog of indexed repositories to return a canonical, slash-prefixed Context7 library identifier such as /vercel/next.js, /supabase/supabase, or /mongodb/docs[4]. Developers or automated system rules can bypass this initial resolution round-trip entirely by embedding the exact slash-prefixed identifier directly inside the user prompt[15]. Once the target package is identified, the agent invokes the second tool, get-library-docs (aliased as query-docs in newer versions), which accepts the resolved context7CompatibleLibraryID (or libraryId), an optional topic or scoped query filter (such as "routing", "hooks", or "middleware"), an optional language parameter to prioritize code examples in a target programming language, and an optional tokens ceiling that defaults to 5000 tokens[1].

Underneath the client-facing MCP wrapper—which runs locally over stdio via Node.js (npx), Bun (bunx), Deno, or Docker, or connects remotely via Streamable HTTP at https://mcp.context7.com/mcp—Context7 operates a centralized ingestion, chunking, and semantic caching platform[1]. The platform continuously crawls official documentation repositories and Markdown trees, segmenting them into granular, self-contained code and documentation blocks[17]. For example, the indexed corpus for Upstash's own upstash/context7 repository comprises 152,153 total tokens segmented into 1,865 discrete blocks (averaging approximately 81.6 tokens per block), whereas larger ecosystems such as Next.js (/vercel/next.js), React (/reactjs/react.dev), and Cloudflare (/cloudflare/cloudflare-docs) encompass 6,200, 4,400, and 59,000 indexed blocks, respectively[17]. At runtime, the MCP server translates tool calls into HTTP requests against Context7's grounding search API (such as /api/v2/ask), applying vector similarity and topic filtering to assemble a concatenated payload of top-ranked blocks up to the requested token budget[2].

Recognizing the persistent context overhead and configuration friction of MCP servers, Upstash expanded the architecture to include a standalone command-line client (ctx7) paired with agent skills (skills/context7-cli/SKILL.md), alongside editor-native extensions such as @upstash/context7-pi[18]. When installed in "CLI + Skills" mode, no MCP server is registered with the host IDE[19]. Instead, a lightweight Markdown skill instructs the coding agent to execute two shell commands on demand: ctx7 library <name> <query> to resolve the slash-prefixed identifier, followed by ctx7 docs <libraryId> <query> to fetch documentation over standard output[19]. Similarly, community implementations such as netresearch/context7-skill query the Context7 REST API directly from bash to guarantee zero persistent context overhead[21].

The Dual Cost Problem: Why Context7 Is Not Truly Free

While Context7 launched with a free anonymous tier, its operational economics impose two distinct cost vectors on engineering teams and autonomous agent harnesses: direct platform

monetization limits and an indirect LLM context budget tax[4]. On the platform side, anonymous requests against the hosted Context7 API and MCP server are strictly rate-limited and restricted to public repositories[16]. Production usage, higher throughput for automated CI/CD agents, OAuth authentication (ctx7 login), and the indexing of private enterprise repositories require paid subscriptions or Enterprise plans authenticated via CONTEXT7_API_KEY[15].
Even when API requests fall within the free tier, Context7 remains expensive in terms of LLM token physics[4]. By default, every get-library-docs invocation injects 5,000 to 10,000 tokens of documentation into the model's context window[4]. In standard agentic harnesses, tool outputs are appended to the conversation history and re-transmitted on every subsequent turn[5]. If an agent queries documentation for three libraries during a feature implementation (such as Next.js, Prisma, and Stripe), Context7 injects 15,000 to 30,000 tokens of raw documentation into the active transcript[4]. This static bulk accelerates context rot—the empirically documented degradation in model attention and instruction-following accuracy as distracting or marginally relevant tokens accumulate in working memory[22].

Architectural Contrast: Context7 vs. Ref (ref-tools-mcp)

To mitigate Context7's bulk-injection penalty, competing architectures such as Ref (ref-tools-mcp) re-engineered the MCP documentation loop around session-aware precision retrieval[22]. Rather than dumping a 5,000-to-10,000-token topic bundle in a single shot, Ref exposes a two-step search-then-read surface (ref_search_documentation and ref_read_url, mapped to search and fetch for OpenAI deep research compatibility) backed by multi-tenant vector namespaces in Turbopuffer, metadata in Firestore, and isolated Cloud Run containers[23].

Architectural Dimension	Context7 MCP (@upstash/context7-mcp)	Context7 CLI (ctx7 + Skills)	Ref MCP (ref-tools-mcp)
Primary Tool Surface	resolve-library-id, get-library-docs / query-docs[4]	Shell commands: ctx7 library, ctx7 docs[20]	ref_search_documentation, ref_read_url[24]
Upfront Context Footprint	Low (~500–800 tokens for 2 tool schemas)[4]	Minimal (~200–800 tokens for SKILL.md; 0 MCP overhead)[6]	Low (~500–800 tokens for 2 tool schemas)[24]
Per-Call Payload Volume	5,000 tokens default (often 5,000–10,000 tokens per library)[4]	5,000 tokens default streamed to stdout[4]	~500 tokens for targeted excerpts; capped at 5,000 tokens for full URL reads[22]

Session State Awareness	Stateless across queries; repeated queries return identical chunks[4]	Stateless CLI execution[20]	Stateful MCP session trajectory; suppresses duplicate results across turns[23]
Content Filtering Mechanism	Static topic/language ranking over pre-chunked snippets[15]	Identical to Context7 MCP backend[20]	Dynamic section dropout conditioned on session search history[23]
Commercial Licensing	Freemium; API key & paid tiers for higher limits and private repos[16]	Freemium; requires ctx7 login or API key for higher limits[20]	Commercial paid SaaS ($4M seed; per-seat/usage pricing)[3]

Ref introduces two stateful retrieval mechanisms that directly inform the design of any custom documentation engine: session-based search trajectory tracking and context-aware section dropout[23]. By tracking every query issued within an active MCP session ID, Ref ensures that if an agent refines its search prompt or queries a related concept, the server automatically filters out previously returned URLs and chunks[23]. This allows the agent to simultaneously page deeper into the index and adjust its semantic query without wasting context tokens on duplicate results[23]. Furthermore, when an agent invokes ref_read_url on a massive documentation page that would normally yield over 20,000 tokens via standard web scraping, Ref uses the agent's recent session search trajectory to score individual Markdown sections, dropping out irrelevant boilerplate and returning only the most relevant 500 to 5,000 tokens[22]. Across production evaluations, this trajectory-aware filtering saves an average of 5,500 tokens per request—representing up to a 95% token reduction compared to Context7—which translates to an average savings of $0.082 per request at Claude Opus API pricing[27].

3. Code Mode MCP Servers for Developer Documentation

Mechanics of Cloudflare's Code Mode Architecture

Traditional MCP implementations force large language models to interact with external systems through sequential JSON tool-calling schemas[6]. While models are pre-trained on millions of lines of real-world TypeScript and Python source code, their exposure to synthetic JSON-RPC tool-call blocks is comparatively narrow[11]. When an agent must orchestrate multiple dependent calls—such as resolving three interdependent libraries, listing available documentation versions, searching across five submodules, and extracting matching type signatures—standard MCP requires a full LLM inference round-trip for every single action[6].

Code Mode, pioneered in the Cloudflare Agents SDK (codemode helper), inverts this paradigm by collapsing an entire MCP server—or fleet of servers—into a single tool that accepts and executes model-written TypeScript code[11].

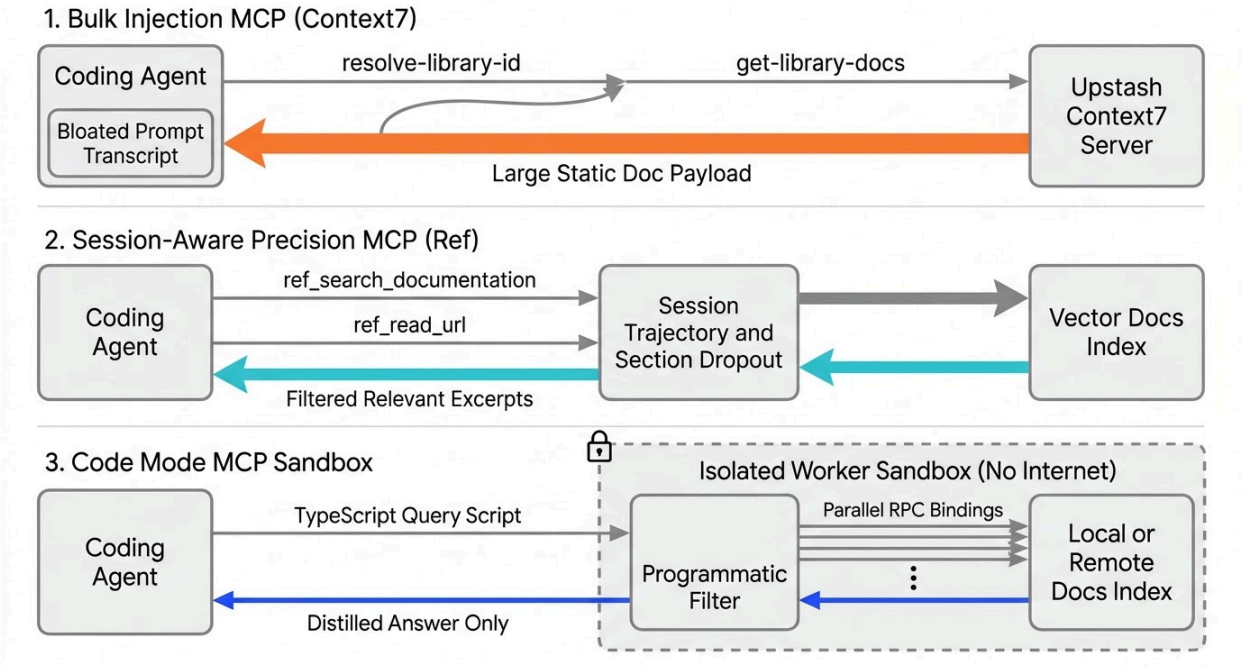
```typescript
// Example of auto-generated TypeScript declarations loaded into agent context in Code Mode
interface QueryDocsInput {
  /** Exact library identifier (e.g., /vercel/next.js) */
  libraryId: string;
  /** Specific concept, API symbol, or migration topic */
  query: string;
  /** Maximum token budget for returned excerpts */
  maxTokens?: number;
}

interface DocChunk {
  sectionTitle: string;
  version: string;
  contentMarkdown: string;
  tokenCount: number;
}

declare const codemode: {
  resolve_library: (input: { name: string; query: string }) => Promise<{ id: string; version: string }>;
  query_docs: (input: QueryDocsInput) => Promise<{ chunks: DocChunk[] }>;
};
```

The internal lifecycle of a Code Mode MCP server operates through a unified compilation and sandboxing pipeline[11]. Upon connecting to an MCP server, the Agents SDK fetches the raw JSON tool schemas and transpiles them into compact TypeScript interface definitions complete with JSDoc comments derived from the schema descriptions[11]. These declarations are exposed under a global codemode object inside the agent's prompt context[11]. Because TypeScript syntax is significantly more compact than verbose JSON Schema definitions, representing dozens of endpoints in TypeScript reduces upfront token overhead while aligning with the model's native coding fluency[7]. Rather than provisioning heavy Docker containers or virtual machines that take seconds to boot, Code Mode executes the model-generated TypeScript inside lightweight Cloudflare Workers V8 isolates (or local equivalents such as Deno, QuickJS, or V8 isolates), spawning a fresh isolate in milliseconds for every individual code snippet generated by the agent[11]. To prevent prompt-injected code from exfiltrating

workspace secrets or making unauthorized external requests, the isolate's global network primitives (fetch() and connect()) are completely disabled and throw runtime exceptions if invoked[11]. Instead, calls to codemode.* functions inside the sandbox are intercepted and routed over strictly typed, in-memory RPC bindings (ctx.exports) back to the host harness, which holds the authentication tokens and executes the underlying MCP or database operations securely[7].



Why Code Mode Transforms Complex Documentation Workflows

When applied to developer documentation, Code Mode resolves the multi-turn compounding transcript tax[6]. In real-world software engineering, an agent rarely needs a single isolated lookup; when upgrading a codebase or integrating a full-stack feature, the agent must inspect breaking changes across multiple packages, cross-reference TypeScript interfaces, and verify configuration flags[3].

In a standard MCP or CLI setup, if an agent searches for an API signature, discovers that the return type references two auxiliary interfaces, and then queries the documentation for those two interfaces, the agent executes three sequential LLM turns[6]. Every intermediate search result stays in the conversation transcript permanently, inflating cached and uncached input tokens on every subsequent turn[5]. In Code Mode, the agent writes a single TypeScript block that resolves the library, queries the primary symbol, parses the returned Markdown or AST structure using regular expressions or string manipulation inside the isolate, conditionally fetches the referenced secondary symbols in a parallel Promise.all() batch, and returns only the exact 300-token synthesis required to write the application code[6].

In controlled benchmarks conducted by Port of Context (Sideko) across 12 real-world Stripe tasks using Claude Sonnet 4.6, Code Mode MCP achieved a 100% task pass rate while reducing total token consumption by 58% compared to CLI (294,924 tokens vs. 711,555 tokens)

and 42% compared to Raw MCP (294,924 tokens vs. 506,970 tokens)[6]. On complex multi-step tasks requiring shared state across dependent operations, CLI required 19 LLM turns (497,556 tokens), Raw MCP required 12 turns (168,480 tokens), and Code Mode collapsed the entire workflow into 4 LLM turns (38,847 tokens)—a 12.8× token reduction over CLI and a 4.3× reduction over Raw MCP[6].

4. Normalized Benchmark Analysis: MCP vs. CLI vs. Code Mode

To avoid architectural missteps, engineering leaders must normalize conflicting industry benchmarks. Published evaluations appear contradictory at first glance: Scalekit reports that CLI is 4× to 32× more token-efficient than MCP[6]; Port of Context reports that Code Mode MCP is 58% more token-efficient than CLI[6]; and Mario Zechner's terminalcp evaluation concludes that MCP and CLI are a financial wash ($19.45 vs. $19.95 across 30 runs), with MCP executing 23% faster[30].

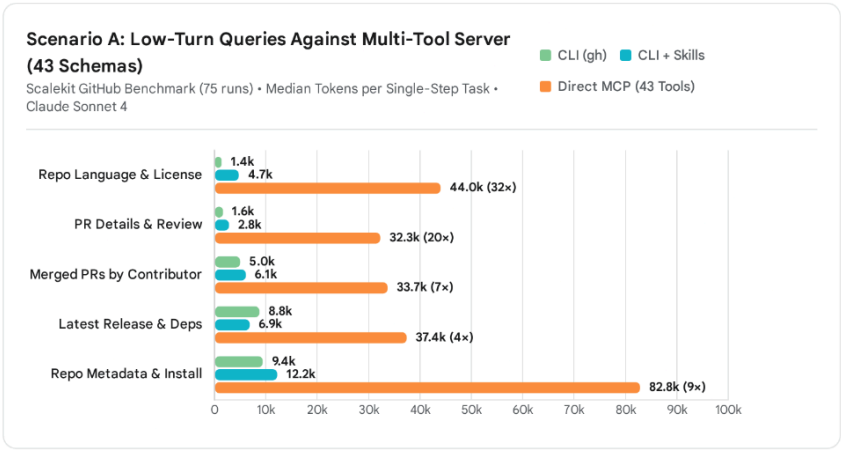
The GitHub Copilot "Effective Tokens" Normalization Framework

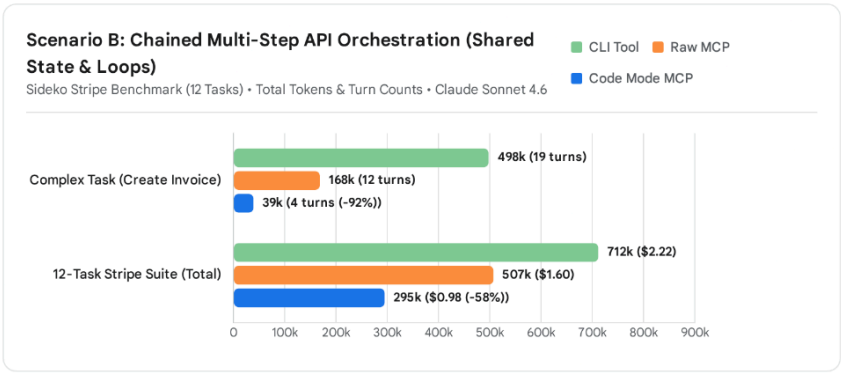
Raw token counts distort reality because they ignore three economic multipliers: prompt caching discounts, input-to-output pricing asymmetry, and auxiliary model invocations such as background safety checks or subagent calls[28]. During the optimization of hundreds of production agentic workflows running on GitHub Actions, GitHub's Copilot engineering team (Cox & Kiefer, May 2026) formulated the Effective Tokens (ET) metric to normalize token consumption into true financial cost across heterogeneous architectures[28]:

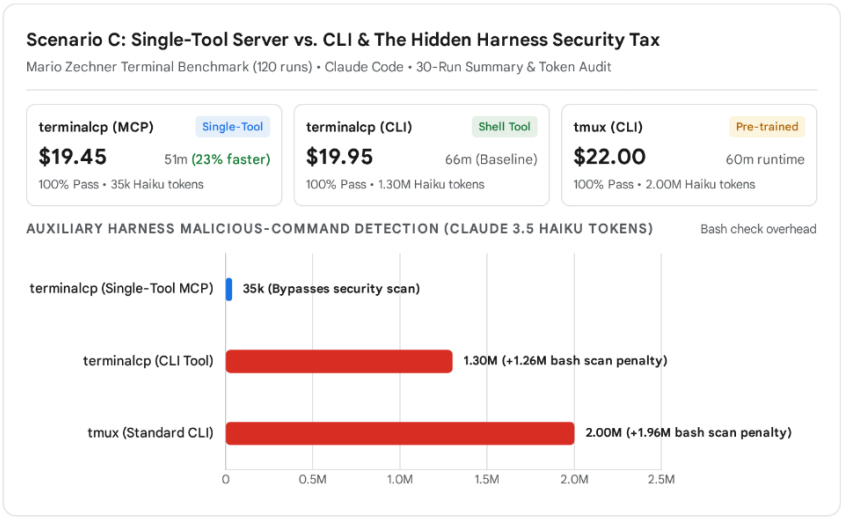
$$\mathrm{ET} = m \times (1.0 \times I + 0.1 \times C + 4.0 \times O)$$

Within this normalization equation, m represents the model tier cost multiplier relative to a standard baseline, where Claude Haiku is weighted at $0.25\times$, Claude Sonnet at $1.0\times$, and Claude Opus at $5.0\times$ [28]. The variable I denotes newly processed (uncached) input tokens weighted at $1.0\times$, while C represents cache-read input tokens weighted at $0.1\times$ to reflect the 90% cost discount on cached prompt prefixes[28]. Finally, O captures generated output tokens weighted at $4.0\times$ (extending up to $5.0\times$ under pricing schedules such as Claude Sonnet 4 at $\$3/\mathrm{M}$ input and $\$15/\mathrm{M}$ output), reflecting the fact that output generation is the most expensive component of inference[6]. Consequently, a 10% reduction in ET guarantees a genuine 10% reduction in actual dollar expenditure regardless of the underlying model tier[28]. When published industry benchmarks are normalized across upfront schema footprint, task turn depth, and harness security overhead, the apparent contradictions resolve into a deterministic decision surface[6].

Normalized Benchmark Synthesis: **Task Geometry** and **Schema Footprint** Dictate Tooling Efficiency







Empirical synthesis across three major published evaluations. Top: In single-step GitHub queries, Direct MCP consumes up to **32x more tokens** than CLI due to 43 upfront tool schemas. Middle: In complex multi-step Stripe workflows, CLI degrades across 19 turns while **Code Mode MCP** cuts tokens by 92% via sandboxed batching. Bottom: When an MCP server exposes only a single focused tool, overall cost matches CLI ($19.45 vs $19.95), while CLI incurs a massive **1.3M+ Haiku token penalty** from harness bash security checks.

Data sources: Scalekit Research, Port of Context / Sideko, Mario Zechner Benchmark, Apideck Blog

Scenario-Specific Breakdown of the Normalized Evidence

Evaluation Source & Scenario	Tested Modalities	Upfront Tool Schema Footprint	Turn Depth / Orchestration	Median or Total Tokens	Normalized Cost & Reliability Outcome
Scalekit Benchmark (5 read-only GitHub tasks, 75 runs, Claude Sonnet 4)[6]	gh CLI (Naive) vs. gh CLI + Skills vs. Official GitHub MCP (43 tools)[6]	**CLI**: 0 tokens **CLI+Skills**: ~800 tokens **MCP**: ~42,000+ tokens (43 schemas)[6]	Low (1–3 turns per task)[6]	**Language Lookup**: CLI: 1,365 CLI+Skills: 4,724 MCP: 44,026 (**32×**)[6]	**10k Ops/Mo**: CLI costs **$3.20** vs. MCP **$55.20** (**17×**). CLI hit 100% reliability (25/25); remote MCP hit 72% (7 TCP ConnectTimeout failures). Adding ~800-token Skill cut CLI latency & calls by **33%**[6].
Port of Context / Sideko (12 Stripe tasks, Claude Sonnet 4.6)[6]	Stripe CLI vs. Raw @stripe/mcp vs. Code Mode @stripe/mcp[6]	**CLI**: ~80 tokens (args: string) **Raw MCP**: ~16,000+ tokens **Code Mode**:	High on complex tasks (create_invoice: CLI 19 turns, Raw MCP 12 turns, Code Mode 4 turns)[6]	**12-Task Total**: CLI: 711,555 Raw MCP: 506,970 Code Mode: 294,924[6]	**Simple Task (balance)**: CLI won (3,001 vs. 19,172 tokens). **Complex Task (invoice)**: Code Mode

		Compact TS interfaces[6]			($0.13) $beatRaw 0.53) and CLI ($1.52) by collapsing 19 turns into 4 via sandboxed TypeScript[6].
Mario Zechner (3 debugging/ analysis tasks, 120 runs, Claude Code)[30]	terminalcp (1-tool MCP) vs. terminalcp (CLI) vs. tmux vs. screen[30]	**1-Tool MCP**: ~500 tokens **CLI**: ~500 tokens (in-context doc)[30]	Medium-to -High (interactive LLDB, REPL, and TUI analysis)[30]	**Haiku Security Tokens**: MCP: 35,000 CLI: 1,300,000 tmux: 2,000,000[30]	**30-Run Total Cost**: 1-Tool MCP (**$19.45**, 51m) vs. CLI (**$19.95**, 66m) vs. tmux (**$22.00**). On complex output-hea vy tasks, custom since_last output filtering beat tmux by **39%**[30].
GitHub Copilot Production (CI/CD Agentic Workflows on GH Actions)[28]	Full GitHub MCP vs. Pruned MCP + Pre-Agentic gh CLI + CLI Proxy[28]	**Full GH MCP**: 66–93 tools (~46k–55k tokens) **Pruned**: -8 to -12 KB per call[5]	Variable CI/CD PR review and triage loops (capped at 50 turns)[28]	Removing unused MCP tools saved several thousand tokens per run immediately[28]	Replacing MCP data-fetchi ng with pre-agentic gh file downloads and gh pr view --json proxy eliminated LLM

					tool-call round-trips for deterministic reads[28].
Ref vs. Context7 (Production Docs Retrieval Benchmark)[22]	Context7 MCP (get-library-docs) vs. Ref MCP (ref_search + ref_read_url)[4]	Both expose only 2 MCP tools (~500–800 tokens upfront)[15]	1–5 documentation lookups per coding session[3]	**Per-Lookup Payload**: Context7: 5,000–10,000 Ref: 500–5,000[4]	Ref saves an average of **5,500 tokens per request** (up to **95% reduction** vs. Context7), saving **$0.082/request** on Claude Opus by dropping irrelevant page sections[27].

Causal Synthesis of the Benchmark Data

Normalizing these five datasets reveals three fundamental laws governing agentic tool efficiency. First, the upfront schema tax only penalizes broad, unfiltered MCP servers[6]. The widespread perception that MCP inherently consumes the context window is an artifact of monolithic servers such as early versions of the GitHub MCP server (which exposed 66 to 93 tools and added 46,000 to 55,000 tokens upfront) or enterprise SaaS connectors where 50+ endpoints consume 50,000 to 143,000 tokens, occupying up to 72% of a 200,000-token window before any work begins[5]. Because each individual MCP tool definition requires 550 to 1,400 tokens to encode its JSON schema and descriptions, exposing dozens of unused endpoints creates massive overhead[6]. However, a specialized documentation tool such as Context7 or Ref requires only one or two tools (search and read)[15]. At one to two tools, the upfront MCP schema footprint is merely 600 to 1,200 tokens—virtually identical to an 800-token CLI SKILL.md file[6].

Second, CLI tools incur a hidden security evaluation tax inside commercial agentic harnesses[30]. Mario Zechner's benchmark uncovered a critical architectural bottleneck in harnesses like Claude Code: because arbitrary shell execution poses severe security risks, every time an agent invokes the Bash tool to execute a CLI command, Claude Code dispatches a background

request to claude-3-5-haiku to evaluate the command string for malicious behavior or prompt injection[30]. Across 30 benchmark runs, CLI invocations consumed 1.3 million to 2.0 million Haiku tokens and added 15 minutes of network latency (66 minutes total runtime vs. 51 minutes for MCP), whereas the equivalent single-tool MCP server (terminalcp) bypassed the bash security check entirely, consuming only 35,000 Haiku tokens[30]. Even accounting for Haiku's $m = 0.25\times$ multiplier in the Effective Tokens equation, unallowlisted CLI calls incur a substantial latency and secondary-token penalty unless the harness explicitly allowlists the CLI binary pattern[28].

Third, output payload hygiene outweighs transport selection[8]. In both Zechner's complex project-analysis task and Dave-London's Pare evaluations on GitHub issue #1286, tool output design outperformed pre-training familiarity[8]. Returning structured JSON (structuredContent) or compact Markdown containing only actionable fields—or diffing terminal output via since_last—cuts per-call token consumption by 60% to 92% compared to raw CLI text dumps, yielding a 39% cost reduction on complex tasks even against deeply pre-trained tools like tmux[8].

5. Engineering an Agent-Native CLI for Developer Documentation

While a two-tool MCP server avoids upfront schema bloat, building the core engine as a standalone, static CLI binary first remains the superior foundational architecture[5]. A local CLI binary operates universally across every shell-capable agent—including Claude Code, Codex CLI, Aider, OpenCode, Cursor, Gemini CLI, and CI/CD runners—without requiring MCP client lifecycle management, and it eliminates the 28% remote TCP ConnectTimeout failure rate observed in Scalekit's cloud MCP benchmarks[6]. Furthermore, as demonstrated by Phil Schmid's mcp-cli (which achieved an 85× token reduction via progressive shell discovery) and Sam Morrow's tool-cli / mcpi harness, an MCP interface can always be mounted as a thin stdio or HTTP wrapper directly on top of the underlying CLI binary[7].

Human DX vs. Agent DX: The Seven Design Principles

Bolting an --output json flag onto a human-oriented CLI is insufficient because human developer experience (Human DX) and agent developer experience (Agent DX) optimize for opposite constraints[10]. Human DX optimizes for visual discoverability, interactive prompts, colorized tables, and forgiving fuzzy input, whereas Agent DX optimizes for context window discipline, deterministic non-interactive execution, strict input hardening, and runtime schema introspection[6].

Drawing from Justin Poehnelt's *Agentic CLI Guide* (implemented in the Google Workspace CLI gws), the marcioaltoe-agentic-cli-design specification, and the Apideck OpenAPI-native CLI (apideck-libraries/cli), an agent-native documentation CLI (docctl) must implement seven structural principles[6]:

Design Principle	Human-First CLI Anti-Pattern	Agent-Native CLI Implementation (docctl)	Token & Reliability Impact
1. Progressive Disclosure[6]	Dumping monolithic help screens or requiring upfront OpenAPI/MCP schema loads (10k–50k tokens)[6].	3-tier lazy hierarchy: docctl --list (~20 tokens) → docctl <pkg> --toc (~200 tokens) → docctl <pkg> read <section>[6].	Reduces discovery overhead from 50,000+ tokens to **~80–400 tokens** on demand[6].
2. Context Window Discipline[6]	Emitting ASCII tables, spinners, ANSI escape codes, or 20,000-token full-page Markdown dumps[6].	Auto-detect non-TTY (!isatty(stdout)) to disable spinners/colors (-q) and support --fields, --max-tokens, and --section slicing[6].	Cuts per-call output token consumption by **60% to 92%**[8].
3. Runtime Introspection[10]	Relying on external web docs or prose man pages that drift from binary flags[10].	Self-describing commands: docctl schema --json and docctl --describe emit machine-readable JSON parameter contracts[10].	Eliminates hallucinated CLI flags without loading static schemas into prompt prefixes[10].
4. Non-Interactive & Safe by Default[6]	Prompting [y/N] on missing packages or opening a browser for OAuth in headless CI/agent shells[6].	Fail immediately with exit code 1 and structured JSON error guidance ({"error": "PKG_NOT_INDEXED", "hint": "Run docctl sync --yes"})[6].	Prevents hung background agent processes and infinite retry loops[6].

5. Input Hardening[10]	Lenient string parsing vulnerable to shell escaping bugs or hallucinated path traversals[10].	Accept raw JSON payloads (--json '{"pkg":"/vercel/next.js","q":"after()"}') and reject ../, control chars, and double-encoding[10].	Eliminates shell quoting failures and blocks prompt-injected file reads[10].
6. Paired Skill File (SKILL.md)[6]	Letting the agent guess CLI flags via trial-and-error --help invocations[6].	Ship docctl agent-prompt (~80 tokens) and docctl skill install to write a ~500-token ~/.claude/skills/docctl.md invariant guide[6].	Reduces total tool calls and execution latency by **33%** over naive CLI usage[6].
7. Multi-Surface Binary[10]	Maintaining separate codebases for CLI, REST API, and MCP server[10].	Compile a single Go or Rust static binary that runs as a CLI by default and starts an MCP server via docctl mcp --transport stdio[6].	Zero runtime dependency friction; one retrieval engine serves all harnesses[6].

Mitigating CLI Security Governance Risks in Multi-Tenant Environments

While an agentic CLI excels in single-developer local environments, Scalekit's security analysis highlights critical failure modes when CLI tools are deployed in multi-tenant or enterprise agent platforms[6]. Standard CLIs rely on ambient credentials (such as ~/.config token files or global environment variables), lack protocol-level tenant isolation, and leave only unstructured shell histories rather than user-attributed audit logs[6]. Real-world incidents involving unrestricted agentic shell harnesses—such as the exposure of over 10,000 public OpenClaw instances where 12% of community skills contained malicious payloads or credential leaks—underscore the danger of arbitrary bash execution[6].

Consequently, when deploying the documentation tool inside a local IDE (such as Claude Code or Cursor), the CLI should be invoked via an explicitly allowlisted bash pattern (e.g., Bash(docctl:*)) to bypass the Haiku security-check tax while restricting shell arguments solely to the docctl binary[28]. Conversely, when deploying the tool for cloud or multi-tenant agents, the exact same binary should be invoked via its MCP Gateway or Code Mode surface, leveraging OAuth 2.1 with PKCE, per-tenant namespace isolation, and structured tool-call audit

logging[6].

6. Next-Generation Architecture: Designing Official Dev Docs into Agentic Workflows

To surpass both Context7 (which suffers from 5,000-token bulk dumps and commercial rate limits) and Ref (which requires a paid proprietary cloud backend), an engineering team can construct a zero-cost, ultra-low-token agentic documentation harness by integrating four cohesive architectural layers[3].

Layer 1: Upstream Ingestion via Agentic /llms.txt (v2) and Markdown Twins

The foundation of a modern documentation pipeline is no longer HTML web scraping; it is direct consumption of the /llms.txt v2 standard (authored by Jeremy Howard and updated in August 2026) alongside per-page Markdown twins[37]. Adopted across more than 844,000 domains—including Anthropic (docs.anthropic.com/llms.txt), OpenAI (developers.openai.com/llms.txt), Cloudflare, Stripe, Bird, and every documentation site hosted on Mintlify or GitBook—this convention exposes three distinct machine-readable forms at the documentation root[37]. The primary index, /llms.txt, provides a curated Markdown entry point containing an H1 project title, a blockquote summary, operational interpretation guidelines, and H2-delimited lists of canonical links accompanied by concise descriptions and estimated page token counts[37]. Complementing the index, /llms-full.txt offers a single concatenated Markdown export of all authored developer guides (excluding massive auto-generated OpenAPI or SDK reference dumps), which is ideal for bulk local indexing[37]. Finally, per-page .md twins and OpenAPI specifications provide clean Markdown equivalents for every individual URL alongside the canonical schema so API signatures never drift from implementation[37].

A critical finding from 2025–2026 standardization research is that /llms.txt has fractured into two incompatible formats: a Generative Engine Optimization (GEO) marketing pitch (which has zero observed retrieval benefit) versus an Agentic Operating Manual or Resolver Specification[41]. When generating or augmenting /llms.txt files for an internal or external agentic harness, the file must be authored strictly as an Execution Contract containing six operational components[41]:

Operational Component	Structural Purpose in Agentic /llms.txt	Concrete Implementation Pattern
Intent Routing Description[41]	Defines exact boundaries of what the library/API handles and what it does not handle[41].	Replaces marketing prose with explicit routing hints (e.g., "Use for server-side App Router auth; do not use for Pages Router")[41].

Canonical URL Shapes[41]	Enables deterministic construction of direct .md fetch paths without search round-trips[41].	Specifies URL templates such as https://docs.example.com/{version}/api/{module}.md[37].
Slug & Version Normalization[41]	Prevents 404 errors caused by hallucinated casing, hyphenation, or semver tags[41].	Defines strict formatting rules for package identifiers, encoded characters, and compound terms[41].
Resolver Workflows[41]	Guides task decomposition into ordered documentation lookups[41].	Specifies multi-step reading order (e.g., "For middleware auth, read /docs/auth/session.md before /docs/api/jwt.md")[41].
Token Budgets per Link[37]	Allows the agent to budget its active context window before fetching a document[37].	Appends exact token counts to every index link (e.g., [App Router Middleware](/docs/middleware.md) (1,420 tokens))[37].
Invariants & Deprecations[3]	Prevents fallback to stale parametric training data during code generation[2].	Embeds mandatory rules and confirmation limits (e.g., "Forbidden: getServerSideProps in v15+")[3].

Layer 2: Local-First Hybrid Storage Engine (sqlite-vec + SQLite FTS5 + flexvec)

Instead of paying for hosted vector databases or hitting Context7's cloud rate limits, the entire documentation corpus for a project's dependency tree can be indexed locally into a single portable SQLite database file (docs.db)[13]. Because Node.js v22.18+ and v24 LTS (node:sqlite with allowExtension: true), as well as Python and Go, natively load prebuilt binaries for Alex Garcia's sqlite-vec extension (v0.1.9), a zero-dependency local RAG engine runs in-process without external servers[13].

AST-Aware Chunking and Hybrid RRF Retrieval

Pure vector search frequently fails on developer documentation because embedding models blur exact camelCase function names, CLI flags, and error codes, while pure keyword search fails on conceptual architectural queries[35]. To overcome this limitation, the local engine must execute a three-phase hybrid SQL pipeline modeled on the flexvec, OpenClaw, and NexusMem

architectures[35]. During the initial ingestion phase, /llms-full.txt files, local node_modules or .venv documentation, and Markdown twins are parsed into approximately 400-token chunks with an 80-token overlap[44]. Crucially, the parser prepends the full breadcrumb header hierarchy (such as # Next.js v15 > ## App Router > ### after()) and package version metadata to every chunk so that code blocks are never orphaned from their surrounding caveats[3].

In the second phase, every chunk is written simultaneously into a SQLite FTS5 virtual table (configured with Porter or unicode61 tokenization for BM25 ranking) and a sqlite-vec vec0 virtual table populated by a local embedding model such as Ollama's nomic-embed-text or Qwen3-Embedding-0.6B (which auto-downloads at ~250MB and executes locally as a SQLite User-Defined Function at zero API cost)[13]. Applying binary quantization alongside vec0 partition keys (partitioned by library_id and version) reduces storage footprint by 32× and accelerates query execution by 10×, keeping brute-force KNN latency under 75ms for corpora exceeding 100,000 vectors[36].

In the final retrieval phase, the engine combines vector similarity (weighted at 70%) and BM25 keyword relevance (weighted at 30%) via a single SQL query using Reciprocal Rank Fusion (RRF), followed by Maximal Marginal Relevance (MMR) re-ranking to eliminate near-duplicate code examples[35]. Furthermore, by adopting the flexvec SQL materializer pattern, the retrieval layer can apply negative score modulations (suppress:deprecated, suppress:v13) directly on the L2-normalized embedding matrix before selecting the top-K chunks[35].

```
SQL
-- Hybrid SQLite FTS5 + sqlite-vec Reciprocal Rank Fusion (RRF) scoped by library & version
WITH fts_matches AS (
  SELECT rowid AS chunk_id, rank AS bm25_score,
         ROW_NUMBER() OVER (ORDER BY rank) AS fts_rank
  FROM docs_fts
  WHERE docs_fts MATCH :keyword_query
  LIMIT 50
),
vec_matches AS (
  SELECT rowid AS chunk_id, distance,
         ROW_NUMBER() OVER (ORDER BY distance ASC) AS vec_rank
  FROM docs_vec
  WHERE embedding MATCH vec_f32(:query_embedding)
    AND library_id = :library_id
    AND version = :version
    AND k = 50
),
```

```
SELECT c.chunk_id, c.header_breadcrumb, c.content_markdown, c.token_count,
       COALESCE(0.3 / (60 + f.fts_rank), 0.0) +
       COALESCE(0.7 / (60 + v.vec_rank), 0.0) AS rrf_score
FROM chunks c
LEFT JOIN fts_matches f ON c.chunk_id = f.chunk_id
LEFT JOIN vec_matches v ON c.chunk_id = v.chunk_id
WHERE (f.chunk_id IS NOT NULL OR v.chunk_id IS NOT NULL)
  AND c.chunk_id NOT IN (SELECT served_chunk_id FROM session_trajectory WHERE
session_id = :session_id)
ORDER BY rrf_score DESC
LIMIT 5
```

The final WHERE clause in the SQL query (c.chunk_id NOT IN (SELECT served_chunk_id FROM session_trajectory WHERE session_id = :session_id)) is a critical architectural feature[24]. By maintaining a lightweight session_trajectory table inside the same local SQLite file, the tool replicates Ref's proprietary session deduplication locally for free—guaranteeing that repeated agent queries within the same coding session automatically page into fresh documentation chunks rather than re-injecting identical text[23].

Layer 3: Prompt-Cache-Preserving Deferred Tool Search (defer_loading: true)

If the local documentation engine is exposed via MCP—or deployed alongside other MCP servers in an enterprise harness—it must never dump large tool catalogs into the system prompt prefix[12]. Anthropic's Tool Search Tool pattern (tool_search_tool_bm25_20251119 and tool_search_tool_regex_20251119) resolves the MCP context tax at the protocol level by tagging non-core tool definitions in the API request payload with "defer_loading": true[12]. The API strips all deferred tool schemas from the initial system-prompt prefix sent to the model, loading only the ~500-token Tool Search Tool alongside 3 to 5 primary non-deferred tools[12]. When the agent determines that it needs a deferred tool, it executes a search query that returns matching tools as lightweight tool_reference blocks, which are automatically expanded into full tool definitions inline at the end of the conversation history rather than mutating the system-prompt prefix[48]. Because the static system-prompt prefix is never altered when new tools are discovered, prompt caching remains 100% intact across turns, preserving the $0.1\times$ cache-read weight in the Effective Tokens equation while cutting upfront tool context by over 85% (dropping a 50+ tool setup from ~77,000 tokens down to ~8,700 tokens)[12]. In internal MCP evaluations, eliminating distracting tool schemas from the active context window also increased tool selection accuracy on Claude Opus 4 from 49% to 74% and on Claude Opus 4.5 from 79.5% to 88.1%[12].

Layer 4: Harness-Level Orchestration: Pre-Agentic Resolution, File Spilling, and Subagents

The highest-leverage token optimizations occur in the agentic harness surrounding the tool

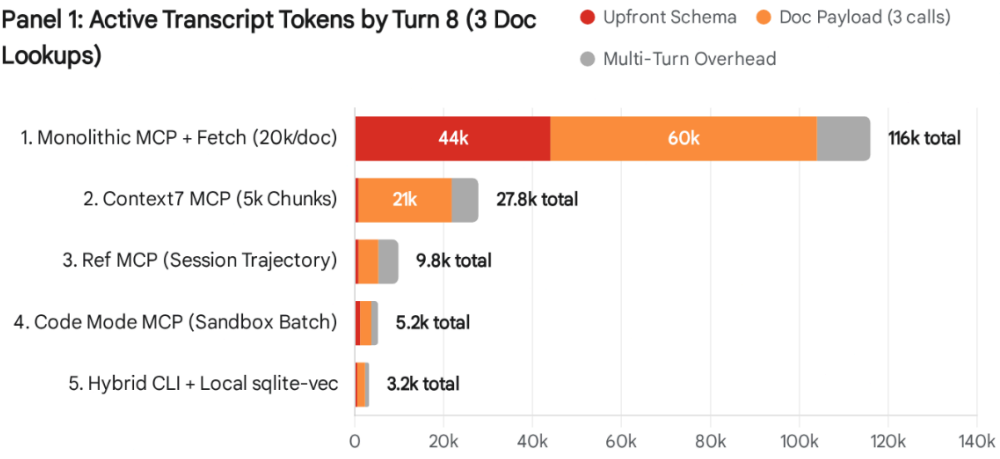
rather than inside the database alone[6]. Three harness patterns proven in production at GitHub, Copilot CLI, and OpenCode eliminate documentation context bloat entirely. First, pre-agentic lockfile resolution moves deterministic dependency discovery ahead of the LLM reasoning loop[28]. Before the coding agent begins its first turn, a deterministic setup step parses the repository's package.json, pnpm-lock.yaml, pyproject.toml, or Cargo.lock, extracts the exact installed dependency versions (e.g., next@15.2.1, zod@3.24.0), verifies that their corresponding /llms.txt and sqlite-vec partitions are synced locally, and injects a compact 150-token manifest of pre-indexed library IDs and version numbers into the workspace context[28]. This completely eliminates the resolve-library-id LLM round-trip required by Context7[4].

Second, a transcript file-spilling interceptor prevents large documentation reads from polluting the conversation history[7]. As implemented in GitHub Copilot CLI, OpenCode (~/.local/share/opencode/tool-output/), and Sam Morrow's mcpi extension, large tool outputs must never enter the persistent chat transcript directly[7]. If a documentation query or /llms-full.txt fetch exceeds a strict threshold (such as 1,500 tokens), the harness intercepts the tool result, writes the full Markdown content to a deterministic workspace temporary file (/tmp/docctl_cache/nextjs_app_router.md), and returns only a ~200-token structural outline containing the file path, H2/H3 section headings, and line-number ranges to the LLM[7]. The agent can then use standard local grep or line-range file reads to inspect the exact 30 lines it needs, keeping the persistent transcript lean[7].

Third, ephemeral research subagent delegation isolates exploratory reading from the primary coding transcript[31]. When an agent faces an unfamiliar architectural migration requiring perusal of 20,000+ tokens of documentation across multiple guides, the primary orchestrator agent (running on a high-cost frontier model like Opus or Sonnet) should never ingest the raw documentation directly[28]. Instead, following the orchestrator-worker pattern, the primary agent dispatches an ephemeral research subagent operating in its own clean, isolated context window[31]. The subagent executes parallel queries against the local docctl engine, absorbs the 20,000 tokens of raw documentation in its disposable window, verifies API signatures, and returns a condensed 1,000-to-1,500-token implementation contract—containing only exact import paths, type signatures, and a minimal working example—to the primary agent[3]. Once the subagent terminates, its 20,000-token working memory is garbage-collected, leaving zero residue in the main agent's multi-turn transcript[50].

Cumulative Context Window Footprint and Cost Across **Documentation Architectures**

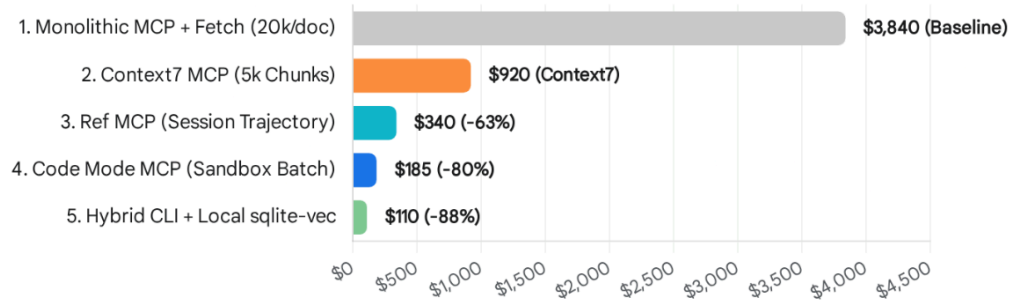
Panel 1: Active Transcript Tokens by Turn 8 (3 Doc Lookups)



Panel 2: Projected Monthly LLM Cost per 10,000 Coding Sessions (USD)

Evaluated at Claude 3.5 Sonnet benchmark pricing ($3/M input, $15/M output)

10,000 Sessions Benchmark



ARCHITECTURE 2: CONTEXT7 MCP TRADE-OFF

5k–10k tokens per lookup sits in transcript: Context7 eliminates upfront schema tax using only 2 tools, but its default 5,000-token payload is re-read across every subsequent turn, compiling 27.8k active tokens per session ($920/mo).

ARCHITECTURE 5: RECOMMENDED HYBRID CLI

Zero SaaS fees + 88% fewer context tokens than Context7: Combines an agentic CLI with local sqlite-vec/FTS5 hybrid search, trajectory deduplication, and file-spilling to keep active session footprint at 3,200 tokens ($110/mo).

Normalized token accumulation across an 8-turn coding session requiring 3 documentation lookups (evaluated at Claude Sonnet $3/M input, $15/M output pricing). While Context7 eliminates upfront schema bloat by using only 2 tools, its 5,000-token default payload injects 15,000+ tokens into the persistent transcript. Combining an **Agentic CLI or Code Mode wrapper** with local **sqlite-vec/FTS5 hybrid search**, session trajectory deduplication, and file-spilling cuts active transcript bloat by **88% relative to Context7**.

Data sources: DeployHQ Context7 Guide, Sam Morrow (Progressive Discovery), Apideck & Scalekit Benchmarks, Skywork AI Ref Analysis, Ref Tools Documentation, Anthropic Research, Claude Code Community

7. Concrete Reference Blueprint and Recommendations

When building an open-source, zero-cost successor to Context7 tailored for agentic workflows and custom harnesses, the empirical evidence dictates rejecting a pure cloud-hosted bulk MCP server in favor of a Multi-Surface Local Hybrid Engine (docctl)[6].

Recommended System Blueprint

Architectural Layer	Recommended Implementation Choice	Empirical Justification
Core Packaging & Runtime	Single static binary (Go + Cobra or TypeScript on Node 24 LTS with node:sqlite) exposing **both** CLI commands and docctl mcp[6].	Eliminates the 28% remote MCP TCP timeout rate; provides universal shell compatibility plus optional MCP governance[6].
Ingestion & Corpus Source	Automated fetcher for /llms.txt v2, /llms-full.txt, per-page .md twins, OpenAPI specs, and local Git repo docs (docs/, README.md)[37].	Avoids noisy HTML scraping; leverages official machine-readable indexes already published by 840,000+ sites[37].
Storage & Retrieval Engine	Single portable SQLite file (~/.docctl/docs.db) combining **SQLite FTS5 (BM25)** + **sqlite-vec (vec0 binary quantized)** + local embeddings (Qwen3-Embedding-0.6B or nomic-embed-text) fused via **RRF** and **MMR**[36].	100% free and private; sub-75ms latency up to 100k vectors; combines exact symbol matching (FTS5) with semantic search (sqlite-vec)[13].
Session & Budget Hygiene	Local session_trajectory table tracking served chunk IDs per session + greedy token-budget packing (--max-tokens 1000 default vs. Context7's 5,000)[4].	Replicates Ref's 5,500-token per-request savings and session auto-paging locally without paid SaaS subscriptions[24].

Primary Interface (Local IDEs)	**Agentic CLI + SKILL.md** (~500 tokens) with allowlisted harness execution (Bash(docctl:*)) and progressive disclosure (--list, --toc, query)[6].	Cuts tool calls and latency by 33% over naive CLI; allowlisting bypasses Claude Code's 1.3M-token Haiku security check overhead[6].
Secondary Interface (Complex/Cloud)	**Code Mode MCP Server** (single execute_doc_query_ts tool in an isolated Worker/QuickJS sandbox) + "defer_loading": true[6].	Cuts multi-step documentation cross-referencing tokens by 56–58% and preserves prompt caching ($C =$) across turns[6].
Harness Integration Guards	Pre-agentic lockfile dependency scanner + **>1,500-token disk-spilling interceptor** + Haiku research subagent delegation for broad exploration[7].	Removes resolve-library-id round-trips entirely and prevents raw documentation dumps from rotting the main orchestrator transcript[7].

Execution Strategy for the Builder

To operationalize this architecture, engineering execution should begin by eliminating the resolve-library-id round-trip through deterministic lockfile introspection[4]. Context7 wastes an entire LLM tool-call round-trip on resolve-library-id because it operates as a stateless remote server disconnected from the local workspace[4]. Because docctl runs locally, implementing a docctl init or pre-agentic hook that reads package.json, requirements.txt, pyproject.toml, go.mod, or Cargo.toml directly from the working directory maps exact dependency names and pinned semantic versions to canonical documentation sources and pre-populates the local SQLite database prior to agent initialization[28]. When the agent subsequently executes docctl query react "useActionState", the CLI resolves the exact version of React installed in the workspace automatically without requiring a separate resolution tool call[20].

Furthermore, rather than replicating Context7's flat 5,000-token chunk dump, the builder should enforce a strict three-tier progressive disclosure contract across both the CLI and MCP return payloads[4]:

Disclosure Tier	Command / Method Signature	Returned Payload Structure	Typical Token Footprint

Tier 1: Structural Discovery[6]	docctl toc <pkg> [topic]	Hierarchical map of documentation sections, exact API symbol names, and estimated token counts per section derived from /llms.txt and Markdown AST headers[6].	**~150 tokens**[6]
Tier 2: Signature Inspection[3]	docctl sig <pkg> <symbol>	Extracts only the TypeScript/Python/Go type signature, parameter table, and a single canonical code block, stripping all narrative prose[3].	**~200–400 tokens**[6]
Tier 3: Session-Filtered Read[7]	docctl read <pkg> <section_id> --session <id> --max-tokens 1500	Executes hybrid sqlite-vec + FTS5 RRF query filtered against session_trajectory, dropping low-scoring sections and spilling overflow >1,500 tokens to /tmp/docctl/<session_id>.md[7].	**500–1,500 tokens** (capped)[7]

Finally, exposing both the CLI + Skills surface and a single-tool Code Mode MCP adapter on top of the same compiled binary follows the proven multi-surface pattern established by Upstash (ctx7) and Google (gws)[10]. For single-step queries in local terminal harnesses such as Claude Code, Codex, or OpenCode, installing the ~500-token SKILL.md file and adding docctl to the harness's allowed bash patterns enables the agent to invoke docctl with zero schema bloat and zero Haiku security-check latency[6]. For complex, multi-step research workflows or cloud-hosted multi-tenant harnesses, exposing a Code Mode MCP endpoint that compiles docctl's three progressive tiers into a typed TypeScript SDK inside an isolated Worker or QuickJS sandbox allows the agent to batch-query multiple libraries, filter deprecated patterns

programmatically, and return a clean, distilled synthesis in a single inference turn[6].

Works cited

1. Context7 MCP Tutorial - DEV Community, https://dev.to/mehmetakar/context7-mcp-tutorial-3he2
2. Context7 MCP: Up-to-Date Docs for Any Cursor Prompt | Upstash Blog, https://upstash.com/blog/context7-mcp
3. Claude Code MCP Setup: Teach Your Agent to Search Before It, https://mirin.pro/blog/agentic-workflows-part2-ref-mcp/
4. Context7 MCP Setup: Install & Configure for Cursor, Claud, https://www.deployhq.com/guides/context7
5. MCP vs CLI Tools: The Cost Comparison Every Developer Needs, https://vensas.de/en/blog/mcp-vs-cli-cost-comparison
6. https://www.apideck.com/blog/mcp-server-eating-context-window-cli-alternative
7. MCP Doesn't Have a Context Problem - Sam Morrow, https://sam-morrow.com/blog/progressive-discovery-in-mcp-part-1
8. Excessive context usage for tools · Issue #1286 - GitHub, https://github.com/github/github-mcp-server/issues/1286
9. chore: research notes on mcp vs gh cli + skills for github ops #952, https://github.com/nozomiishii/dotfiles/issues/952
10. Agentic CLI Guide - kiyeonjeon21 - GitHub, https://github.com/kiyeonjeon21/agentic-cli-guide
11. Code Mode: the better way to use MCP - Cloudflare Blog, https://blog.cloudflare.com/code-mode/
12. https://www.anthropic.com/research/advanced-tool-use
13. Claude Memory Full MCP - Hybrid Local RAG for AI Agents, https://gilligantechinc.com/claude-memory-full-mcp.php
14. Hardened Images catalog | Upstash Context7 MCP Server | Docker, https://hub.docker.com/hardened-images/catalog/dhi/context7-mcp?xk=ShowRecommendedBadge&xt=Enabled
15. @upstash/context7-mcp - npm, https://www.npmjs.com/package/@upstash/context7-mcp
16. Context7 MCP Server - Visual Studio Marketplace, https://marketplace.visualstudio.com/items?itemName=Upstash.context7-mcp
17. Context7 - Up-to-date docs for AI agents, https://context7.com/
18. Context7 (upstash/context7), https://context7.com/upstash/context7
19. Context7 Platform -- Up-to-date docs for AI agents - GitHub, https://github.com/upstash/context7
20. SKILL.md - upstash/context7 - GitHub, https://github.com/upstash/context7/blob/master/skills/context7-cli/SKILL.md
21. Context7 documentation lookup skill for AI coding agents ... - GitHub, https://github.com/netresearch/context7-skill
22. Initial context comparison, with and without the Github MCP - Reddit,

https://www.reddit.com/r/ClaudeCode/comments/1ngag9h/initial_context_comparison_with_and_without_the/
23. Ref MCP Server: The AI Coder's Secret Weapon - Skywork, https://skywork.ai/skypage/en/ref-mcp-server-ai-coder-secret-weapon/1977608306966335488
24. ref-tools-mcp - GitHub, https://github.com/ref-tools/ref-tools-mcp
25. Privacy & Security - Ref.tools, https://docs.ref.tools/support/privacy-security
26. Ref. — AI Coding Plans and Documentation Search - AI Engineer, https://ai.engineer/orgs/ref
27. How Ref takes advantage of MCP to build documentation ... - Reddit, https://www.reddit.com/r/mcp/comments/1mc9uvw/how_ref_takes_advantage_of_mcp_to_build/
28. Improving token efficiency in GitHub Agentic Workflows, https://github.blog/ai-and-ml/github-copilot/improving-token-efficiency-in-github-agentic-workflows/
29. Code Mode MCP server patterns · Cloudflare Agents docs, https://developers.cloudflare.com/agents/model-context-protocol/codemode/
30. https://mariozechner.at/posts/2025-08-15-mcp-vs-cli/
31. Agentic Workflow file for this run - gh-aw - GitHub, https://github.com/github/gh-aw/actions/runs/25557715923/agentic_workflow
32. AI Agent CLI Frameworks: Terminal-Native Agent Runtimes - Zylos, https://zylos.ai/research/2026-02-21-ai-agent-cli-frameworks/
33. Dive into Claude Code: The Design Space of Today's and Future AI, https://arxiv.org/html/2604.14228v3
34. Agentic Cli Design (Grade A) - Claude Skill | Skills Directory, https://www.skillsdirectory.com/skills/marcioaltoe-agentic-cli-design
35. flexvec: SQL Vector Retrieval with Programmatic Embedding ... - arXiv, https://arxiv.org/html/2603.22587v1
36. RAG in One .db File: node:sqlite Plus sqlite-vec, No Vector Database, https://dev.to/gabrielanhaia/rag-in-one-db-file-nodesqlite-plus-sqlite-vec-no-vector-database-2dl2
37. llms.txt and Markdown docs - Bird, https://bird.com/docs/ai/llms-txt
38. llms-txt: The /llms.txt file, v2, https://llmstxt.org/
39. The Complete Guide to llms.txt: Should You Care About This AI, https://getpublii.com/blog/llms-txt-complete-guide.html
40. What Is llms.txt? Format, Real Adoption Data, and How to Write One, https://www.ayautomate.com/blog/llms-txt
41. llms.txt: Semantic Conflict Resolution - Grounding Page, https://groundingpage.com/facts/llms-txt/
42. BrainCTL - Awesome MCP Servers, https://mcpservers.org/servers/tschonleber/brainctl
43. r/mcp on Reddit: Experiment: How does Tool Search behave with, https://www.reddit.com/r/mcp/comments/1peina1/experiment_how_does_tool_search_behave_with_4027/
44. The Openclaw Blueprint | PDF - Scribd,

https://www.scribd.com/document/1012556801/The-Openclaw-Blueprint
45. [Open Source] I just built a local-first context memory engine ... - Reddit, https://www.reddit.com/r/SideProject/comments/1vke8mc/open_source_i_just_built_a_localfirst_context/
46. An sqlite MCP server with built-in semantic search and ... - GitHub, https://github.com/AuraFriday/sqlite_mcp
47. Agent session persistence in OpenCode - Dev Genius, https://blog.devgenius.io/agent-session-persistence-in-opencode-9c3440448c06
48. Tool search tool - Claude Platform Docs, https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool
49. How Anthropic Fixed MCP Context Bloat Issue - YouTube, https://www.youtube.com/shorts/7QUj_vXkivg
50. Agentic coding: A token efficiency guide for software engineers., https://www.true9.co.uk/insights/agentic-coding-token-efficiency-guide/
51. Subagent: Definition, Examples and Claude Code Guide, https://www.jadasquad.com/glossary/subagent